

Longest common subsequence between run-length-encoded strings: a new algorithm with improved parallelism

Valerio Freschi, Alessandro Bogliolo *

STI, Università di Urbino, 61029 Urbino, Italy

Received 30 September 2003; received in revised form 3 February 2004

Communicated by A.A. Bertossi

Abstract

Data compression can be used to simultaneously reduce memory, communication and computation requirements of string comparison. In this paper we address the problem of computing the length of the longest common subsequence (LCS) between run-length-encoded (RLE) strings. We exploit RLE both to reduce the complexity of LCS computation from $O(M \times N)$ to $O(mN + Mn - mn)$, where M and N are the lengths of the original strings and m and n the number of runs in their RLE representation, and to improve the inherent parallelism of the proposed algorithm, so that it may execute in $O(m + n)$ steps on a systolic array of $M + N$ units.

We also discuss the application of the proposed algorithm to the related problem of edit distance (ED) computation.

© 2004 Elsevier B.V. All rights reserved.

Keywords: Algorithms; Longest common subsequence; Run-length encoding; String comparison; Parallel processing

1. Introduction

String comparison is a critical task in many applications, ranging from molecular biology to data mining. The aim of string comparison is the evaluation of a given similarity/distance metric for a pair of strings defined over a common finite alphabet. Given a string S , any string obtained by deleting some of the characters of S is said to be a subsequence of S . Given two strings X and Y , a common subsequence for X and Y is a string that is a subsequence of both of them.

The length of the *longest common subsequence* (LCS) of X and Y provides a widely used measure of similarity between the two strings.

A related distance metric is the *edit distance* (ED), defined as the minimum cost of the edit operations required to transform X in Y , or viceversa. The basic edit operations are insertion, deletion and substitution of single characters. The cost of a sequence of operations is computed as the sum of the costs of the elementary operations it involves (C_{ins} , C_{del} , C_{sub}). Since there are many operation sequences that can be applied to transform X in Y , the edit distance is defined as the minimum transformation cost computed over all possible solutions.

* Corresponding author.

E-mail address: alessandro.bogliolo@uniurb.it (A. Bogliolo).

When $C_{\text{sub}} \geq C_{\text{ins}} + C_{\text{del}}$, computing ED and LCS are instances of the same problem. In fact, given two strings X and Y of M and N characters respectively, the following relation holds:

$$\begin{aligned} \text{ED}(X, Y) &= C_{\text{ins}}(N - \text{LCS}(X, Y)) \\ &\quad + C_{\text{del}}(M - \text{LCS}(X, Y)). \end{aligned} \quad (1)$$

From a computational stand point, LCS can be exactly computed by means of dynamic programming [5] in a number of steps proportional to the product of the lengths of the two strings: $O(NM)$. Advanced algorithms have been proposed solving LCS with complexity: $O(pN + N \log N)$ and $O(p(M + 1 - p) \log N)$ [9] (where p is the length of the LCS), $O((r + N) \log N)$ [10] (where r is the total number of matching pairs of X and Y), $O(N \log |\Sigma| + d \log \log \min\{d, MN/d\})$ [8] (where d is the number of so-called dominant matches, and $|\Sigma|$ is the cardinality of the alphabet), or $O((M + N)|\Sigma| + |M| \log |M|)$ [3] (where $|M|$ is the cardinality of the set of so-called maximal matches between substrings of X and Y). The bit-level parallelism (W) of the microprocessor that executes the algorithm has been also exploited to achieve complexity $O(MN/W)$ [6].

Finally, the inherent parallelism of the basic dynamic programming algorithm has been exploited to perform LCS/ED computation in $O(M + N)$ steps [12]. The straightforward parallel implementation makes use of a systolic matrix of $M \times N$ identical computational units with local connections. However, the number of units that actually execute in parallel is limited to $\min\{M, N\} + 1$, so that parallel implementations have been proposed that make use of only $\min\{M, N\} + 1$ shared units [12].

We remark that $O(M + N)$ is not only the computational time required by hardware accelerators, but also the amount of memory required to store the two strings and the time required to load them from memory. Hence, memory and communication issues are as critical and computational issues in order to speed up LCS/ED evaluation [11].

In principle, memory, communication and computation needs could be simultaneously reduced by taking advantage of string compression techniques. In particular, several algorithms have been recently proposed that exploit either run-length encoding (RLE) [13] or Lempel–Ziv compression (LZ78) [14]. The

complexity achieved by using RLE is either $O(mN + Mn)$ [2,4] or $O(mn \log(mn))$ [1], where m and n are the numbers of *runs* (i.e., maximum substrings of consecutive characters taking the same value) in X and Y , while the complexity achieved for LZ78 is $O(MN(1/\log N + 1/\log M))$, that reduces to $O(N^2/\log N)$ when $M = N$ [7].

Existing approaches, however, suffer from two main limitations. First, they reduce the asymptotic complexity at the cost of increasing the number of operations required at each step, thus achieving small benefits in most practical cases. Second, they focus only on computational complexity, reducing the inherent parallelism that could be exploited by hardware accelerators.

In this paper we propose a new $O(mN + Mn - mn)$ algorithm for LCS computation between RLE strings with improved parallelism. Our algorithm can be executed in $O(m + n)$ steps by a linear array of elementary units performing only sums and comparisons.

2. Background

The basic dynamic programming algorithm for computing the LCS between two strings (namely, $X = x_1, x_2, \dots, x_M$ and $Y = y_1, y_2, \dots, y_N$) can be mapped on a matrix (hereafter denoted by *LCS*) of $M + 1$ rows and $N + 1$ columns. Rows 1 to M are associated with the elements of X , while columns 1 to N are associated with the elements of Y . The first row and column, with index 0, are associated with no elements and are initialized to 0. Entry $\text{LCS}(i, j)$ represents the LCS between the first i characters of X and the first j characters of Y , and can be incrementally computed from $\text{LCS}(i - 1, j)$, $\text{LCS}(i - 1, j - 1)$ and $\text{LCS}(i, j - 1)$ by means of the following recursive equation:

$$\text{LCS}(i, j) = \begin{cases} 0, & \text{if } i = 0 \text{ or } j = 0, \\ \max\{\text{LCS}(i - 1, j), \text{LCS}(i, j - 1)\}, & \text{if } x_i \neq y_j \text{ and } i, j > 0, \\ \text{LCS}(i - 1, j - 1) + 1, & \text{if } x_i = y_j \text{ and } i, j > 0. \end{cases} \quad (2)$$

Entry $\text{LCS}(M, N)$, computed in $M \times N$ steps, represents the LCS between X and Y .

A *run* is a sequence of consecutive characters taking the same value. The run-length is the number

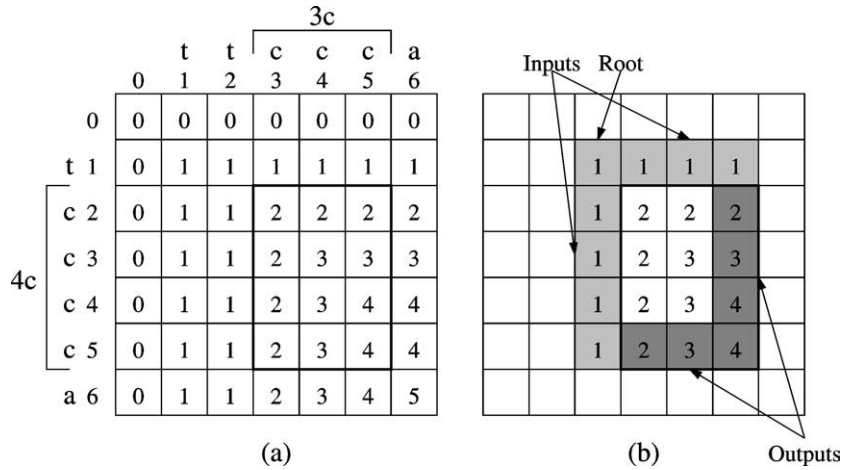


Fig. 1. (a) Example of a 7×7 LCS matrix, (b) 4×3 block associated with runs 4c of X and 3c of Y . Shading is used to highlight input, output and root elements.

of characters in the run. RLE is a string-compression technique that represents a string as a sequence of runs rather than as a sequence of characters [13]. For instance, string “acccttgagg” can be represented as “1a,4c,2t,3g”, reducing the number of elements from 10 characters to 4 runs.

The runs of X and Y induce a partitioning of the LCS matrix into submatrices associated with ordered pairs of runs of the two strings. We call *block* each submatrix B of LCS associated with a pair of runs of X and Y . With respect to a block B we call: *input elements* the entries of LCS that do not belong to B but provide inputs to some of its elements, *root element* the input element with smallest row and column indexes, *output elements* the elements of B that feed entries of LCS that do not belong to the block, *inner elements* the elements of B that are not output elements. Submatrix $LCS(2 : 5, 3 : 5)$ in Fig. 1 is a block, its input elements are $LCS(1, 3 : 5)$, $LCS(2 : 5, 2)$ and $LCS(1, 2)$, that is also the root, its output elements are $LCS(5, 3 : 5)$ and $LCS(2 : 4, 5)$, while its inner elements are $LCS(2 : 4, 3 : 4)$.

In a straight-forward implementation of recursive equation (2), the comparison between symbol ‘g’ of string X and symbol ‘c’ of string Y is performed 12 times to update the entries of block $LCS(2 : 5, 3 : 5)$. Moreover, at least 6 steps (in case of a parallel implementation) are required to move across the block by incrementally computing all its entries.

If X and Y are RLE strings, however, symbol repetitions are explicitly represented and can be exploited to save computation effort and time because:

1. the comparison between symbols associated with two runs can be performed once for all,
2. inner elements of a block do not need to be computed,
3. all output elements of a block can be updated in parallel (i.e., all data dependencies among them can be removed).

In the following section we provide a formal proof of the above statements and we use them to derive a new algorithm for LCS that exploits RLE both to reduce computational complexity and to improve inherent parallelism.

3. The proposed approach

We exploit the monotonicity of the LCS matrix, that is formally stated by the following lemma.

Lemma 1 (Monotonicity). *Given a LCS matrix, the value in position (i, j) is greater or equal than all values in the submatrix $LCS(0 : i, 0 : j)$. In symbols:*

$$LCS(i, j) \geq LCS(h, k) \quad \forall (h, k) \mid h \leq i, k \leq j.$$

Proof. $LCS(i, j)$ represents the LCS between the first i elements of X , denoted by $X(1 : i)$, and the first j elements of Y , denoted by $Y(1 : j)$. Similarly, $LCS(h, k)$ is the LCS between $X(1 : h)$ and $Y(1 : k)$. If $h \leq i$ and $k \leq j$, $X(1 : h)$ is a substring of $X(1 : i)$ and $Y(1 : k)$ is a substring of $Y(1 : j)$. Hence, the longest common subsequence between $X(1 : h)$ and $Y(1 : k)$ is also a common subsequence of $X(1 : i)$ and $Y(1 : j)$. As a consequence, $LCS(i, j) \geq LCS(h, k)$. $\square$

According to the result of the comparison between the symbols associated with a run of X and with a run of Y , we denote the corresponding block of the LCS matrix as either *match block* or *mismatch block*. The following two theorems provide the basis for computing the output elements of a match/mismatch block directly from the values of the input elements, without looking at the inner elements.

Theorem 1 (Match). *The value of the output element $LCS(i, j)$ of a match block rooted in $(i0, j0)$ is given by*

$$LCS(i, j) = LCS(i - d, j - d) + d,$$

where $d = \min\{i - i0, j - j0\}$.

Proof. From recursive equation (2) we observe that whenever $x_i = y_j$, entry $LCS(i, j)$ is obtained by adding 1 to $LCS(i - 1, j - 1)$. This corresponds to a diagonal move on the matrix. In case of a match block, d is the number of diagonal moves across the block leading to output element $LCS(i, j)$. Since the LCS is incremented at each move in the original dynamic programming algorithm, the result of the d moves across the match block can be directly computed by adding d to the value of the input element leading to $LCS(i, j)$ along a diagonal path. $\square$

Theorem 2 (Mismatch). *The value of the output element $LCS(i, j)$ of a mismatch block rooted in $(i0, j0)$ is given by*

$$LCS(i, j) = \max\{LCS(i, j0), LCS(i0, j)\}.$$

Proof. Mismatch blocks do not contribute to the LCS. Hence, any path across a mismatch block does not change the value of the input element it starts from.

The value to be assigned to output element $LCS(i, j)$ is nothing but the maximum value found among the input elements that may lead to that output. In principle, any input element $LCS(i0, k)$ with $j0 \leq k \leq j$, or $LCS(h, j0)$ with $i0 \leq h \leq i$ may lead to $LCS(i, j)$. However, the search for the maximum can be restricted to $LCS(i0, j)$ and $LCS(i, j0)$ because of monotonicity. Hence, $LCS(i, j)$ can be computed looking only at 2 input elements. $\square$

The above theorems prove that any output element of a block of the LCS matrix can be computed in constant time based on 1 or 2 input elements of the same block. On the other hand, input elements are either boundary elements (initialized to 0) or outputs of other blocks, so that the only elements of the LCS matrix that need to be computed are the outputs of the blocks. This observation leads to an $O(mN + Mn - mn)$ algorithm.

Theorem 3 (Complexity). *The LCS between two RLE strings X (with M characters and m runs) and Y (with N characters and n runs) can be computed in $O(mN + Mn - mn)$.*

Proof. According to Theorems 1 and 2, LCS computation entails $O(1)$ processing of each output element of the blocks associated with any pair of runs of X and Y . The overall complexity of the algorithm is given by the total number of output elements, that can be computed as the product of the number of blocks (mn) times the average number of output elements per block (given by the average run length of X , M/m , plus the average run length of Y , N/n , minus 1, subtracted to avoid counting twice the corner output element). Hence

$$O\left(mn\left(\frac{M}{m} + \frac{N}{n} - 1\right)\right) = O(mN + Mn - mn). \quad \square$$

The pseudo-code of the RLE-LCS algorithm is reported in Fig. 2. For the sake of simplicity, we assume LCS matrix to be already allocated and initialized to 0. Blocks are visited row by row, by means of two nested loops that span the runs of X and Y . For each block, an inner loop is used to span the output elements. Within the loop, the values of the corresponding runs are first compared to distinguish between match and mismatch cases, then the output elements are updated according

```

for each run x of X {
  for each run y of Y {
    B = block associated with runs x and y;
    i0 = row of the root element of B;
    j0 = column of the root element of B;
    for each output element o of B {
      i = row of o;
      j = column of o;
      if value of x = value of y {    // match
        d = min{i-i0, j-j0};
        LCS(i,j) = LCS(i-d,j-d) + d;
      } else                          // mismatch
        LCS(i,j) = max{LCS(i0,j), LCS(i,j0)};
    }
  }
}

```

Fig. 2. RLE-LCS algorithm for computing LCS between RLE strings in $O(mN + Mn - mn)$ steps.

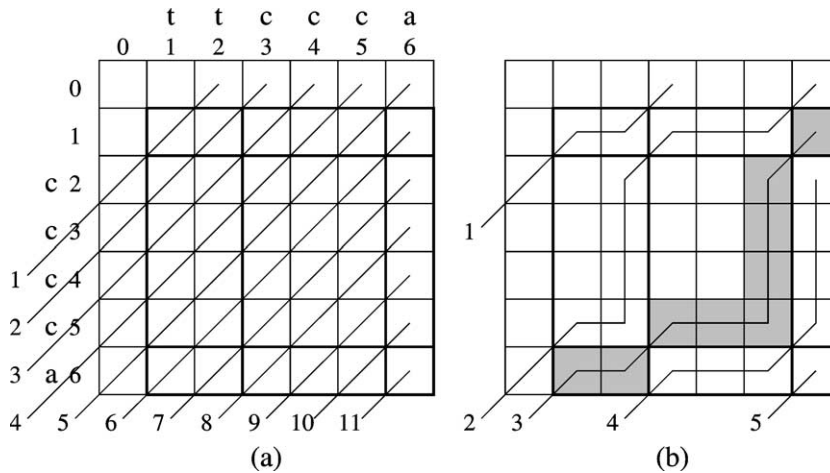


Fig. 3. Steps involved in the parallel computation of a 7×7 LCS matrix according to (a) the traditional dynamic programming algorithm and (b) the proposed algorithm for RLE strings.

either to Theorem 1 or to Theorem 2. Second-order code optimizations are not shown in Fig. 2 for the sake of readability. In particular, since the values of x and y do not change within each block, the comparison could be performed once per block before entering the inner loop.

3.1. Parallelism

Fig. 3(a) shows the execution steps of a parallel systolic implementation of the basic dynamic pro-

gramming algorithm for computing LCS between two strings of 6 characters each. Initialization of the first row and column is not accounted for. The maximum number of units that execute in parallel is 6 (i.e., $\min\{M, N\}$) and the entire execution requires 11 steps (i.e., $M + N - 1$). As remarked in Section 2, the computation of all entries of submatrix $LCS(2 : 5, 3 : 5)$ takes 6 steps.

The execution steps of a parallel implementation of the RLE-LCS algorithm are shown in Fig. 3(b) for the same strings. As expected, the inner elements of the

blocks are never visited, and all the output elements of each block are computed in parallel, since there are no data-dependencies among them. Hence, the maximum number of units that can execute in parallel is 9, execution completes in only 5 steps, and a single step is required to jump across submatrix $LCS(2 : 5, 3 : 5)$.

In general, the number of steps is given by the sum of the runs of the two strings, so that the execution time is $O(m + n)$, with a speedup proportional to the compression factor of RLE with respect to a systolic implementation of the basic LCS algorithm. Notice that a similar speedup is achieved in terms of transfer time from memory, so that the reduction in computation time is partially sustained by the reduction of transfer time [11].

The number of units that work in parallel ranges from $\min\{M, N\}$ (in the worst case of non-RLE strings) to $M + N$ (in the best, but trivial, case of strings composed of a single run). Hence, a systolic array of $M + N$ shared elements is required to fully exploit the maximum achievable parallelism of RLE-LCS.

Although addressing hardware implementation issues is behind the scope of this work, we remark that the actual parallelism of RLE-LCS may change at every step, depending on the size of the blocks under processing. This complicates the control logic of the parallel implementation, but it does not impair performance, since it does not impact the complexity of the basic operations performed by each unit.

3.2. Application to ED

The RLE-LCS algorithm proposed in this paper can be used to compute the ED between two RLE strings whenever ED computation reduces to LCS computation. The applicability is guaranteed by Eq. (1) whenever edit operations are assigned with costs such that

$$C_{\text{sub}} \geq C_{\text{ins}} + C_{\text{del}}. \quad (3)$$

It can be shown that the above condition is not only sufficient, but also necessary to the application of the algorithm of Fig. 2 to ED computation. The formal proof is omitted here for the sake of conciseness.

Although an $O(mN + Mn - mn)$ algorithm for ED was proposed by Bunke et al. [4], the application of RLE-LCS to ED provides a two-fold contribution. First, RLE-LCS has a wider range of application than

the algorithm by Bunke et al., that was derived only for $C_{\text{sub}} = 2$, $C_{\text{ins}} = C_{\text{del}} = 1$. Second, it has improved parallelism, that enables the computation of ED in $O(m + n)$ steps by means of parallel hardware.

4. Conclusion

In this paper we have presented a new algorithm for the computation of LCS between RLE strings that achieves complexity $O(mN + Mn - mn)$, where M and N are the lengths of the original strings and m and n the number of runs in their RLE representation. The proposed approach exploits RLE not only to reduce the computational complexity, but also to improve the parallelism of the basic dynamic programming algorithm for LCS, so that it can be executed in $O(m + n)$ steps by a systolic array of $M + N$ units.

The proposed approach can be applied to the related problem of ED computation whenever the costs assigned with edit operations satisfy

$$C_{\text{sub}} \geq C_{\text{ins}} + C_{\text{del}}.$$

References

- [1] A. Apostolico, G. Landau, S. Skiena, Matching for run-length encoded strings, *J. Complexity* 15 (1) (1999) 4–16.
- [2] O. Arbell, G.M. Landau, J.S.B. Mitchell, Edit distance of run-length encoded strings, *Inform. Process. Lett.* 83 (2002) 307–314.
- [3] B.S. Baker, R. Giancarlo, Sparse dynamic programming for longest common subsequence from fragments, *J. Algorithms* 42 (2002) 231–254.
- [4] H. Bunke, J. Csirik, An improved algorithm for computing the edit distance of run length coded strings, *Inform. Process. Lett.* 54 (1995) 93–96.
- [5] T.H. Cormen, C.E. Leiserson, R.L. Rivest, *Introduction to Algorithms*, MIT Press, Cambridge, MA, 1990.
- [6] M. Crochemore, C.S. Iliopoulos, Y.J. Pinzon, J.F. Reid, A fast and practical bit vector algorithm for the longest common subsequence problem, *Inform. Process. Lett.* 80 (2001) 279–285.
- [7] M. Crochemore, G.M. Landau, M. Ziv-Ukelson, A sub-quadratic sequence alignment algorithm for unrestricted cost matrices, in: *Proceedings of ACM-SIAM Symposium on Discrete Algorithms*, 2002, pp. 679–688.
- [8] D. Eppstein, Z. Galil, R. Giancarlo, G. Italiano, Sparse dynamic programming I: Linear cost functions, *J. ACM* 39 (1992) 519–545.
- [9] D.S. Hirschberg, Algorithms for the longest common subsequence problem, *J. ACM* 24 (4) (1977) 664–675.

- [10] J.W. Hunt, T.G. Szymanski, A fast algorithm for computing longest common subsequences, *Comm. ACM* 20 (1977) 350–353.
- [11] T. Kahveci, A. Singh, Accelerating substring searching: breaking the I/O barrier, UCSB Technical Report, 2002.
- [12] D. Lopresti, P-NAC: a systolic array for comparing nucleic acid sequences, *Computer* 24 (11) (1987) 6–13.
- [13] K. Sayood, E. Fow (Eds.), *Introduction to Data Compression*, second ed., Morgan Kaufmann, Los Altos, CA, 2000.
- [14] J. Ziv, A. Lempel, Compression of individual sequences via variable-rate coding, *IEEE Trans. Inform. Theory* II-24 (1978) 530–536.